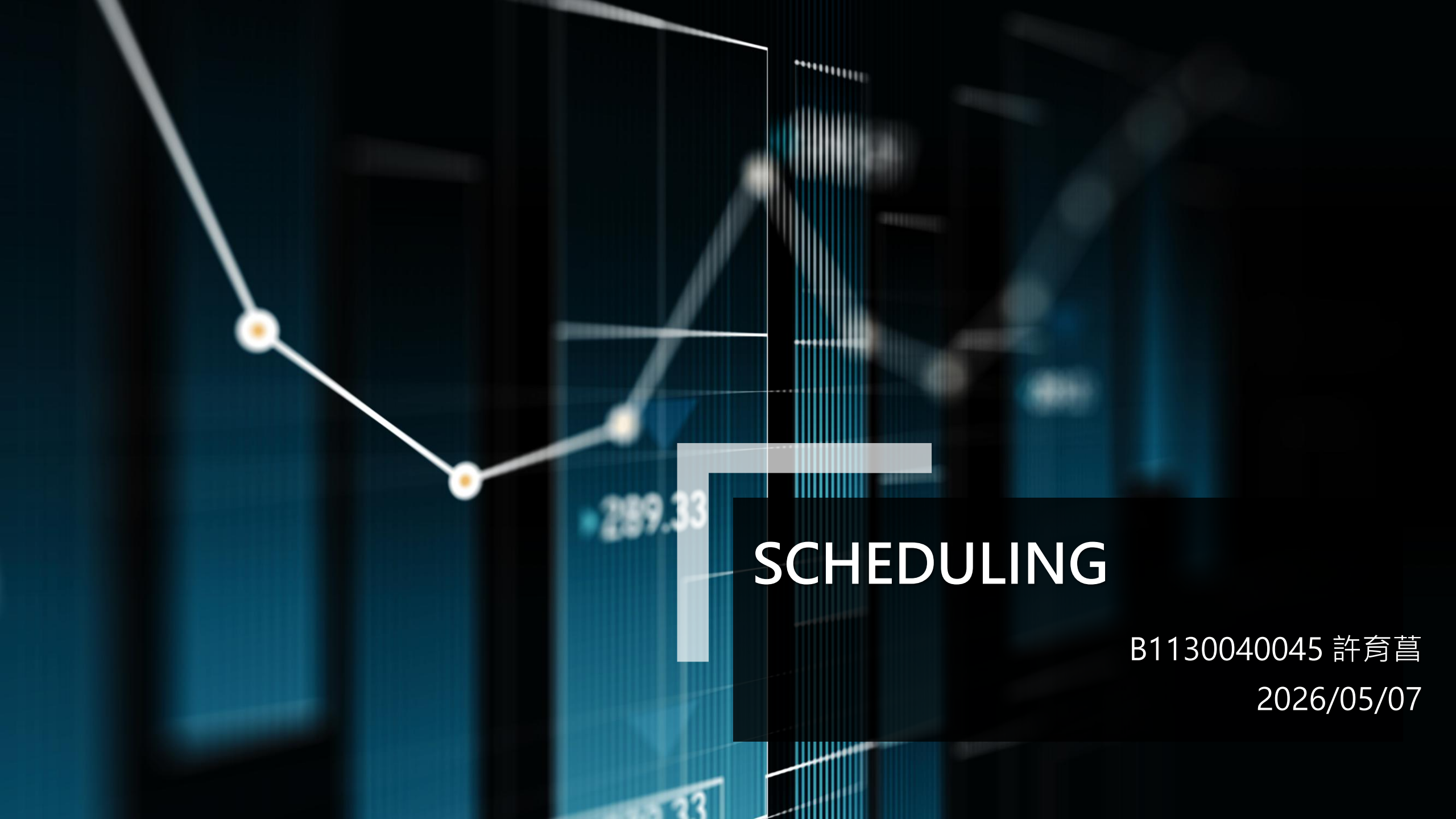


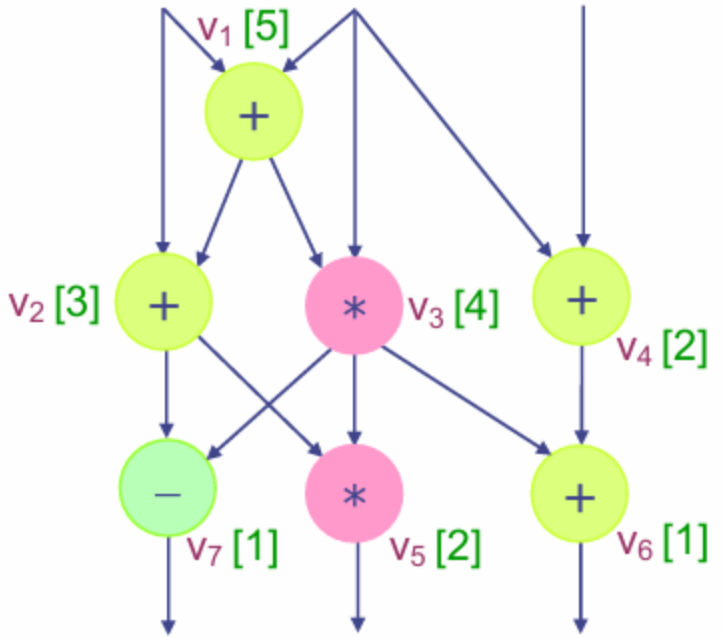
SCHEDULING

B1130040045 許育菖

2026/05/07

題目說明

- 給定一連串的operation，每個operation可能使用其他operation的輸出作為參數。我們需要在有限adder以及multiplier的情況下，使用list scheduling來在最少的時間(step)內完成所有運算。



INDEX	op	參數1	參數2	輸出
0	1	1	2	10
1	1	10	3	12

在左圖，operation 1需要使用operation 0的輸出作為參數，因此operation 1要在operation 0完成後才能執行。

DFG Scheduling 程式邏輯

1、 讀檔，為每個operation建立一個node，並以vector來儲存所有的node。

2、在建立node的同時，使用一個陣列來記錄「當前operation輸出對應當前operation的index」。(若index為-1，代表可以取得這個參數)

```
int producer[MAX_NODE]; // the node value can get from which operation.
                        // index: node, value: operation
int producerSaved[MAX_NODE];

// initialize to -1, which means this node is an outside input
for(int i = 0; i < MAX_NODE; i++) producer[i] = -1;


// input operation from file
for(int i = 0; i < op; i++){
    int op, node1, node2, result;
    in >> op >> node1 >> node2 >> result;

    producer[result] = i;
    list.push_back( Operation(op, node1, node2, result) );
}
```

DFG Scheduling 程式邏輯

3、 從最後一個operation開始，藉由剛剛紀錄的「operation輸出對應operation的index」，以DFS的方式遍歷整個list，依序為每個operation標記critical path priority。

可以透過從後往前遍歷，以及「若當前node以儲存的level小於等於遍歷的level」就回傳，來提高DFS的效率。

```
// build critical path priority
for(int i = op-1; i >= 0; i--){
    int level = 1;
    DFS(producer, list[i].node1, level+1);
    DFS(producer, list[i].node2, level+1);
}
```

```
// build crtical path priority
void DFS(const int* producer, int node, int level){
    if(producer[node] == -1) return;

    if(list[ producer[node] ].level >= level) return;

    list[ producer[node] ].level = level;

    DFS(producer, list[ producer[node] ].node1, level+1);
    DFS(producer, list[ producer[node] ].node2, level+1);
}
```

DFG Scheduling 程式邏輯

4、 接著進入主迴圈，每次遍歷整個list，選擇當前可以執行的operation將其放入ready queue，並**按照第3步建立的critical path priority由大到小進行排序**。

然後根據可使用的adder和multiplier數量，**從ready queue中提取operation執行**。並**更新「operation輸出對應operation的index」的值為-1**，因為這個node已經可以被使用了。

5、重複第4步直到ready queue和list皆為空，完成list scheduling！

RBG to YUV程式邏輯

與DFG scheduling幾乎相同，只是在紀錄「當前operation輸出對應當前operation的index」使用map<string, int>來儲存，而不是int陣列。

```
map<string, int> producer;

for (int i = 0; i < opCount; i++) {
    int op;
    string node1, node2, result;

    in >> op >> node1 >> node2 >> result;

    producer[result] = i;
    listOp.push_back(Operation(op, node1, node2, result));
}
```

實驗結果

DFG1

(細節請見output)

```
Which file do you want to open?
        1) DFG1.txt
        2) DFG2.txt
        3) other~
1
[ALU Resource]
- Multiplier: 1
- Adder: 1
  1 pick: (  1) + (  2) = ( 10)
  2 pick: (  6) + (  7) = ( 11)
  3 pick: ( 10) + (  3) = ( 12)
  4 pick: ( 11) + (  8) = ( 13)
  5 pick: ( 12) + (  4) = ( 14)
  6 pick: ( 13) + (  9) = ( 15)
  7 pick: ( 14) + ( 15) = ( 16)
  8 pick: ( 16) * ( 16) = ( 17)
  9 pick: ( 17) + ( 12) = ( 19)   ( 16) * ( 16) = ( 18)
 10 pick: ( 12) + ( 19) = ( 20)
 11 pick: ( 18) + ( 15) = ( 23)   ( 20) * ( 20) = ( 21)
 12 pick: ( 23) + ( 15) = ( 25)
 13 pick: ( 10) + ( 21) = ( 26)   ( 25) * ( 25) = ( 27)
 14 pick: ( 26) + ( 19) = ( 29)
 15 pick: ( 10) + ( 26) = ( 28)
 16 pick: ( 27) + (  9) = ( 32)   ( 28) * ( 28) = ( 30)
 17 pick: ( 29) + (  5) = ( 31)
 18 pick: ( 23) + ( 32) = ( 34)   ( 31) * ( 31) = ( 33)
 19 pick: ( 32) + (  9) = ( 35)
 20 pick: ( 19) + ( 16) = ( 22)   ( 35) * ( 35) = ( 38)
 21 pick: (  1) + ( 30) = ( 36)
 22 pick: ( 11) + ( 34) = ( 37)
 23 pick: ( 33) + (  5) = ( 40)   ( 37) * ( 37) = ( 41)
 24 pick: ( 22) + ( 23) = ( 24)
 25 pick: ( 32) + ( 38) = ( 43)
 26 pick: ( 36) + ( 26) = ( 39)
 27 pick: ( 31) + ( 40) = ( 42)

[ALU Resource]
- Multiplier: 1
- Adder: 2
  1 pick: (  1) + (  2) = ( 10)   (  6) + (  7) = ( 11)
  2 pick: ( 10) + (  3) = ( 12)   ( 11) + (  8) = ( 13)
```

DFG2

(細節請見output)

```
Which file do you want to open?
        1) DFG1.txt
        2) DFG2.txt
        3) other~
2
[ALU Resource]
- Multiplier: 1
- Adder: 1
  1 pick: (169) * (169) = (  3)
  2 pick: (169) * (169) = ( 11)
  3 pick: (169) * (169) = (  2)   (  3) + ( 11) = ( 19)
  4 pick: (169) * (169) = (  4)
  5 pick: (169) * (169) = ( 10)
  6 pick: (169) * (169) = ( 12)   (  2) + ( 10) = ( 18)
  7 pick: (169) * (169) = ( 27)   (  4) + ( 12) = ( 20)
  8 pick: (169) * (169) = (  5)   ( 19) + ( 27) = ( 35)
  9 pick: (169) * (169) = ( 13)
 10 pick: (169) * (169) = ( 26)   (  5) + ( 13) = ( 21)
 11 pick: (169) * (169) = ( 28)   ( 18) + ( 26) = ( 34)
 12 pick: (169) * (169) = ( 43)   ( 20) + ( 28) = ( 36)
 13 pick: (169) * (169) = (  1)   ( 35) + ( 43) = ( 51)
 14 pick: (169) * (169) = (  6)
 15 pick: (169) * (169) = (  9)
 16 pick: (169) * (169) = ( 14)   (  1) + (  9) = ( 17)
 17 pick: (169) * (169) = ( 29)   (  6) + ( 14) = ( 22)
 18 pick: (169) * (169) = ( 42)   ( 21) + ( 29) = ( 37)
 19 pick: (169) * (169) = ( 44)   ( 34) + ( 42) = ( 50)
 20 pick: (169) * (169) = ( 59)   ( 36) + ( 44) = ( 52)
 21 pick: (169) * (169) = ( 25)   ( 51) + ( 59) = ( 67)
 22 pick: (169) * (169) = ( 30)   ( 17) + ( 25) = ( 33)
 23 pick: (169) * (169) = ( 45)   ( 22) + ( 30) = ( 38)
 24 pick: (169) * (169) = ( 58)   ( 37) + ( 45) = ( 53)
 25 pick: (169) * (169) = ( 60)   ( 50) + ( 58) = ( 66)
 26 pick: (169) * (169) = ( 74)   ( 52) + ( 60) = ( 68)
 27 pick: (169) * (169) = ( 41)   ( 67) + ( 74) = ( 80)
 28 pick: (169) * (169) = ( 46)   ( 33) + ( 41) = ( 49)
 29 pick: (169) * (169) = ( 61)   ( 38) + ( 46) = ( 54)
 30 pick: (169) * (169) = ( 73)   ( 53) + ( 61) = ( 69)
 31 pick: (169) * (169) = ( 75)   ( 66) + ( 73) = ( 79)
 32 pick: (169) * (169) = ( 86)   ( 68) + ( 75) = ( 81)
 33 pick: (169) * (169) = (  7)   ( 80) + ( 86) = ( 92)
```

RGB to YUV

(細節請見output)

```
[ALU Resource]
- Multiplier: 1
- Adder/Subtractor: 1
  1 pick:     R * 0.299 =   (1)
  2 pick:     G * 0.587 =   (2)
  3 pick:     R * 0.169 =   (6)     (1) +   (2) =   (4)
  4 pick:     G * 0.331 =   (7)
  5 pick:     B *   0.5 =   (8)     (6) +   (7) =   (9)
  6 pick:     R *   0.5 =  (12)     128 +   (8) =  (10)
  7 pick:     G * 0.419 =  (13)     (9) -  (10) =     U
  8 pick:     B * 0.081 =  (14)    (12) -  (13) =  (15)
  9 pick:     B * 0.114 =   (3)     128 -  (14) =  (16)
 10 pick:   (4) +   (3) =     Y
 11 pick:  (15) +  (16) =     V
Total control steps: 11

[ALU Resource]
- Multiplier: 1
- Adder/Subtractor: 2
  1 pick:     R * 0.299 =   (1)
  2 pick:     G * 0.587 =   (2)
  3 pick:     R * 0.169 =   (6)     (1) +   (2) =   (4)
  4 pick:     G * 0.331 =   (7)
  5 pick:     B *   0.5 =   (8)     (6) +   (7) =   (9)
  6 pick:     R *   0.5 =  (12)     128 +   (8) =  (10)
  7 pick:     G * 0.419 =  (13)     (9) -  (10) =     U
  8 pick:     B * 0.081 =  (14)    (12) -  (13) =  (15)
  9 pick:     B * 0.114 =   (3)     128 -  (14) =  (16)
 10 pick:   (4) +   (3) =     Y    (15) +  (16) =     V
Total control steps: 10

[ALU Resource]
- Multiplier: 1
- Adder/Subtractor: 3
  1 pick:     R * 0.299 =   (1)
  2 pick:     G * 0.587 =   (2)
  3 pick:     R * 0.169 =   (6)     (1) +   (2) =   (4)
  4 pick:     G * 0.331 =   (7)
  5 pick:     B *   0.5 =   (8)     (6) +   (7) =   (9)
  6 pick:     R *   0.5 =  (12)     128 +   (8) =  (10)
  7 pick:     G * 0.419 =  (13)     (9) -  (10) =     U
```

實驗心得

我覺得這次的實驗還滿好玩的，可以讓我認識到怎麼去排程，之前只在書上看到排程相關的演算法，直到這次實驗才第一次動手實作。

但我認為實驗題目解釋有點偏模糊，尤其是RGB to YUV的部分，因為題目只給一張圖片，第一反應很容易就往輸入RGB三個參數，按照公式得到YUV來想。差一點就白白浪費時間寫一堆用不到的程式碼。這也不禁讓我再次理解到理解題目在資工領域中才是最重要的！努力不如用對方法！